SESSION 2

From ML To GenAI In Energy

Agentic AI For Petroleum Engineers: From LLM Foundations To Workflow Automation

Session 2 explains how modern LLMs evolved, how they become chat and reasoning systems, how agents and coding copilots work, and how these tools can be applied in practical petroleum-engineering workflows.

Learning Path

This session connects model fundamentals to practical engineering productivity.

1. **LLM History**
2. **Tokens & Embeddings**
3. **Alignment**
4. **Reasoning**
5. **Model Ecosystem**

1. **Agents**
2. **Coding Copilots**
3. **Oil & Gas Workflows**
4. **Hands-On Forecasting Demo**

> The goal is not to memorize product names. The goal is to understand what these systems are, what they can reliably do, and where human review still matters.

Read more: [Hugging Face LLM Course](#) · [IEA: Energy And AI](#)

A Short History Of Modern LLMs

2017

Attention Is All You Need introduces the transformer architecture and changes NLP.

2018

Models like BERT show that transformer pretraining can produce strong reusable language representations.

2020

GPT-3 demonstrates few-shot prompting at large scale.

2022

ChatGPT popularizes instruction-following conversational systems.

2023–Today

Claude, Gemini, new GPT families, and open-weight models turn LLMs into multi-modal assistants, coding copilots, and agents.

Google

Helped create the transformer era.

OpenAI

Helped commercialize large-scale chat and reasoning assistants.

Read more: Attention Is All You Need · BERT · Language Models Are Few-Shot Learners

Why Attention Changed Everything

The Problem Before Attention

Earlier sequence models struggled to retain long-range relationships efficiently.

What Attention Does

- Lets the model decide which earlier tokens matter most for the current prediction.
- Makes it easier to represent context, parallelize training, and scale to larger datasets.

What A Transformer Is Not

A transformer is not a database or a physics model. It is a **learned pattern engine** that uses attention to build context-sensitive representations.

Read more: [Attention Is All You Need](#) · [Hugging Face Transformers Docs](#)

Practical Takeaway

Better context handling is one reason LLMs became useful for:

- Summarization
- Code generation
- Retrieval workflows
- Reasoning-like tasks

What Tokens Are

Models do not read raw language the way humans do. They process **tokens**, which are small text chunks such as whole words, parts of words, punctuation, or code fragments. Tokenization is a compression and encoding step that turns text into model-readable units.

Example Input

`Forecast oil rate for Well A next month`

Possible token split: `Forecast` `oil` `rate` `for` `Well` `A` `next` `month`

Code Is Even More Token-Heavy

`df["oil_rate"] = oil_bbl / days_on_prod`

Symbols and fragments increase token count significantly.

↑ Tokens

More Context Used

Context windows, latency, and cost are all strongly affected by token count.

↑ Latency

Slower Response

More tokens means more compute per request.

↑ Cost

Higher Cost

Good prompting means being precise enough to guide the model, but efficient enough not to waste context.

Read more: [Hugging Face Tokenizers Docs] · [OpenAI tiktoken Cookbook]

How Tokens Become Embeddings

Each token is mapped to a dense numeric vector called an **embedding**. Embeddings place semantically related text closer together in vector space. Inside a transformer, embeddings are repeatedly updated through many layers so meaning becomes more context-aware.

Simple Mental Model

01

Tokens

The model's input pieces

02

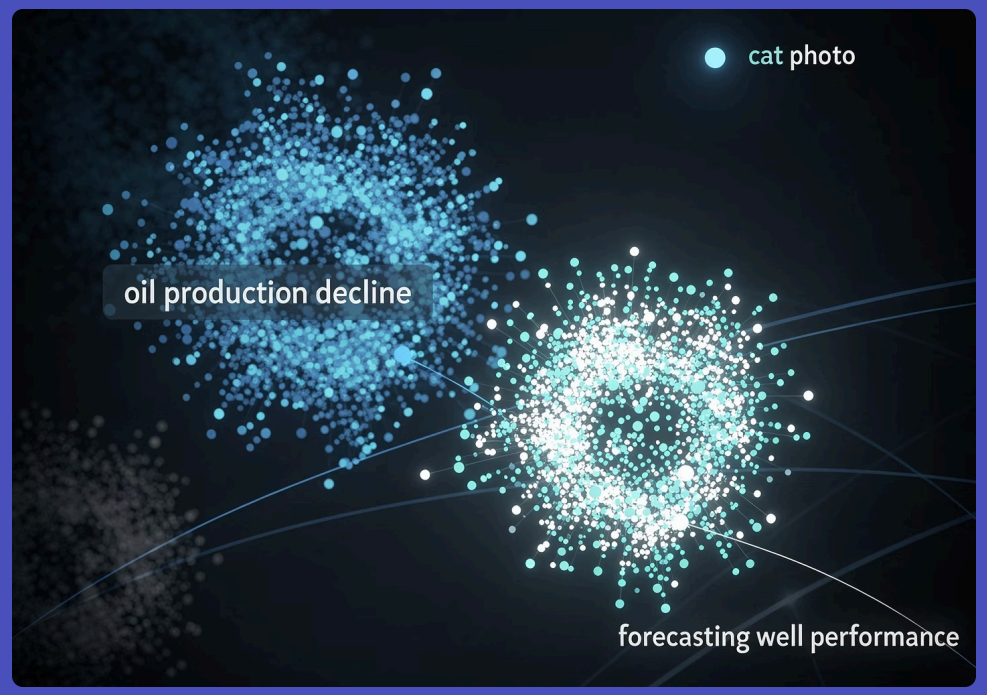
Embeddings

Their numeric representations

03

Attention

Updates those representations based on context



Vector Space Example

Close together:

- oil production decline
- forecasting well performance

Far away:

- cat photo

Similar meaning → closer vectors. That is why embedding search can retrieve semantically related engineering text even when the wording is not identical.

This is why embeddings are central to **RAG**, semantic search, and document comparison systems.

Read more: [OpenAI Embeddings Guide] · [Introducing Text And Code Embeddings]

How Base Models Are Trained

Training Process

- A base model is usually pretrained on massive text and code corpora.
- The core objective is often **next-token prediction**.
- The model adjusts billions of parameters so it becomes better at predicting the next token across many contexts.
- It does not memorize every source as a clean library.
- It learns statistical patterns, structures, style, associations, and latent representations.

What Pretraining Produces

- A model with broad, general-purpose language understanding
- The ability to complete, continue, and reason about text across many domains
- Latent knowledge of facts, code patterns, writing styles, and logical structures
- No specific persona, safety guardrails, or instruction-following behavior yet

That is why pretrained models can write, summarize, translate, answer questions, and generate code before any domain-specific tuning.

Important Caution: Pretraining produces broad capability. It does not automatically produce safe, helpful, or grounded chat behavior.

Read more: [Language Models Are Few-Shot Learners] · [Hugging Face LLM Course]

How Base Models Become Chat Assistants

Instruction Tuning

Teaches the model to follow user requests in a more usable format.

RLHF

Uses ranked human preferences to improve helpfulness and reduce harmful or low-quality behavior.

Other Alignment Approaches

Preference optimization and rule-based feedback methods such as Constitutional AI.

Raw base models are powerful but not necessarily aligned for helpful conversation. This is how a broad language model becomes a more reliable assistant for chat, drafting, and workflow support.

> **Practical Takeaway:** Chat models are not just bigger base models — they are base models plus alignment layers, policy choices, and post-training optimization.

Read more: [InstructGPT / RLHF Paper] · [The FLAN Collection] · [Constitutional AI]

Reasoning Models

Standard Chat Models

Optimized for fast, general responses.

Reasoning Models

Optimized to spend more compute on harder tasks such as planning, multi-step math, code repair, or structured analysis. They often perform better when the task needs decomposition, verification, or tool-supported iteration.

> They are not magic and they are not always the right default. They usually **trade speed and cost for deeper problem solving**.

When Reasoning Models Help

- Multi-step engineering calculations
- Code debugging
- Structured planning
- Analysis where intermediate constraints matter

Read more: Chain-of-Thought Prompting · OpenAI Reasoning Models Guide

Flagship Models Versus Open-Source Models

Flagship Hosted Models

Usually the top hosted families from vendors such as OpenAI, Anthropic, and Google.

Strengths: frontier capability, reliability, long context, strong tooling, and managed infrastructure.

Open-Source / Open-Weight Models

Offer more transparency, local deployment options, and customization flexibility.

Examples: Llama (Meta), Mistral, Qwen (Alibaba), DeepSeek, Falcon, Phi (Microsoft), Gemma (Google)

The Right Choice Depends On:

- Capability
- Privacy
- Latency
- Cost
- Deployment
- Governance

> **Common Pattern:** Hosted flagship models for highest capability and managed operations. Open-weight models for experimentation, on-prem constraints, or domain customization.

Read more: OpenAI Platform Overview · Anthropic Docs Overview · Gemini API Docs · Hugging Face Transformers Docs

How To Access Models In Practice

API Platforms

Give access to hosted models over HTTP and SDKs. Usually easier for production-grade capability.

Hugging Face

Provides model discovery, documentation, datasets, inference options, and open-source ecosystem tools.

Ollama

Makes it much easier to run supported models locally for experimentation. Useful for privacy, offline work, or lower-cost experimentation.

Decision Questions

- Do you need the strongest model quality?
- Do you need local execution?
- Do you need GPU infrastructure?
- Do you need enterprise logging, rate control, and access management?

Read more: Hugging Face · Ollama Docs · Gemini Quickstart

What An AI Agent Is

Chatbot

A chatbot answers a prompt. It takes a single input and returns a single response. The interaction ends there.

- Single turn: ask once, answer once
- Some memory of prior steps
- No tool use or external actions
- Fast and simple

Agent

An agent tries to complete a task. That usually means multiple steps, decisions, tool calls, memory of prior context, and some notion of state or policy.

- Multi-step task execution
- Can call tools and APIs
- Maintains state across steps
- Useful when ask once, answer once is not enough

The point is not autonomy for its own sake — it is reliable task completion across connected steps.

Read more: [OpenAI Agents SDK Guide] · [Anthropic Building With Claude]

Agent Anatomy

Model

The brain that interprets instructions and generates actions or text.

Context

System prompt, files, history, and retrieved evidence.

Tools

APIs, databases, code execution, search, calculators, or file operations.

Memory / State

What the system remembers during the task.

Policy

What is allowed, what needs approval, and when to stop.

Output Contract

The expected structure, artifacts, or decisions.

Healthy Agent Design Principles

Narrow Scope

Explicit Tool Schemas

Observable Steps

Human Review At Decision Points

Read more: [OpenAI Platform Overview](#) · [Anthropic API Docs](#)

Tool Calling With A Tiny Example

An agent becomes more useful when it can call tools instead of pretending to know everything.

```
def get_latest_oil_rate(well_id: str) -> float:
    # tool implementation hidden from the model
    ...

question = "What is the latest oil rate for WELL_14?"

# model decides:
# 1. call get_latest_oil_rate("WELL_14")
# 2. read the returned number
# 3. answer with the result and context
```

What Matters

- The tool has a **clear name**
- The **input schema is constrained**
- The model can only call **allowed tools**
- The result can be **logged and reviewed**

Read more: [Function Calling With Gemini](#) · [OpenAI Agents SDK Guide](#)

MCP And Standardized Tool Connectivity

What Is MCP?

MCP stands for Model Context Protocol. It is a standard way to expose tools, resources, and prompts to models and agents.

The value is not only integration speed. The value is **cleaner boundaries** between the model and the external systems it can use.

In practice, MCP can connect agents to files, databases, APIs, browser tools, and internal services in a structured way.

Read more: MCP Specification · Build An MCP Server

Why It Matters

- Less ad hoc wiring
- Clearer schemas
- Easier tool reuse
- Better interoperability across agent frameworks and IDEs

AI-Assisted Coding: From Autocomplete To Agents

Autocomplete A Line

The first wave: small code suggestions.

Generate A Function

Tools like GitHub Copilot expand into chat, inline explanation, refactoring help, and code generation.

Explain & Refactor Files

Understand and restructure existing code across files.

Agentic Coding System

Planning, repo awareness, terminal use, file edits, test execution, and multi-step coding workflows. The shift from autocomplete assistant to agentic coding system.

Read more: [GitHub Copilot Docs](#) · [Concepts For GitHub Copilot](#)

Why Cursor, Claude Code, And Codex Matter

Cursor

One of the strongest AI-native IDE experiences because it combines codebase awareness, chat, editing, agent modes, terminal execution, and repo-scale workflows in one environment.

Claude Code

Pushes toward terminal-first, agentic coding workflows with strong code reasoning and task execution patterns.

OpenAI Codex

Represents OpenAI's direction for code generation and agentic software tasks.

Why Cursor stands out for this course: Easy for students to install and use · Strong support for file-aware chat and code edits · Supports prompt-driven workflow building without heavy setup · Useful bridge between notebook user and agent builder

Frontier coding tools are unlocking faster prototyping, debugging, refactoring, and workflow automation for non-software specialists.

Read more: Cursor Introduction · Cursor Agent Modes · Claude Code Overview · OpenAI Codex Docs

What This Unlocks In Oil And Gas

Petroleum engineers spend large amounts of time on repetitive technical glue work. AI coding and agent tools can compress that work dramatically when the task is reviewable. The productivity gain is often not from replacing engineering judgment — it is from **accelerating data cleanup, scripting, documentation, comparison, reporting, and workflow packaging**.

High-Value Examples

Convert Surveillance Logic

Into reusable Python

Generate SQL

For production or well-event pulls

Compare Offset Wells

And summarize differences

Draft Review Notes

For reserves or forecasting

Build Dashboards

For technical screening

Operationalize Notebooks

Turn analysis into a repeatable internal helper

Read more: [IEA: AI For Energy Optimisation And Innovation] · [SPE: Unlocking The Potential Of Generative AI In Oil And Gas]

Production Forecasting Hands-On

Follow-Along Path

1. Start with `training/hands-on/production-forecasting-demo.md`
2. Run `training/hands-on/production-forecasting-demo.ipynb`
3. Use either `production-forecasting-colab-guide.md` or `production-forecasting-local-setup.md`
4. Review the supporting data files under `training/sample-data/production_forecasting/`
5. Optionally continue the AI-copilot extension in the standalone repo: **hands-on-ml-cursor**

Technical Framing

- Compare **Arps hyperbolic decline** against **RandomForestRegressor**, **Exponential smoothing**, and **ARIMA**
- Use 30 months train, 6 months test, and 12 months forecast
- Use the LLM to help package, explain, and operationalize the workflow after the analysis is produced

Recap And Additional Reading

01

LLMs Are The Foundation

02

Alignment Makes Them Usable For Chat

03

Reasoning Models Improve Hard-Task Performance

04

Agents Add Tools, State, And Multi-Step Execution

05

Coding Copilots Bring This Into Daily Engineering Work

06

Petroleum Engineers Can Accelerate High-Friction Technical Workflows Without Giving Up Domain Accountability

Additional Reading

- Attention Is All You Need
- Hugging Face LLM Course
- OpenAI Platform Docs
- Anthropic Docs
- Gemini API Docs
- MCP Specification
- Cursor Docs
- IEA: Energy And AI